

Notification-Oriented Paradigm Framework 2.0: An Implementation Based On Design Patterns

A. F. Ronszcka, G. Z. Valença, R. R. Linhares, J. A. Fabro, P. C. Stadzisz and J. M. Simão

Abstract— The Notification-Oriented Paradigm (NOP) is a new technique to develop software. NOP is a rule-oriented approach where every rule and fact-base element is derived into less complex entities with specific tasks. These entities particularly collaborate by means of notifications, which occur only when their state changes. This allows carrying out an inference process completely active. Due to this particular arrangement, NOP can eliminate most of the structural and temporal redundancies that affects program execution performance, difficulties in codification level, and high coupling in program modules. In order to validate the NOP state of art, a framework was initially implemented in C++. Even though it quite demonstrated the features of the paradigm in terms of the development process, it still presented a gap in execution performance. In this context, this paper presents the NOP Framework 2.0. This new version was reengineered aiming better structuration and improvements in the execution time of NOP applications. The experiments show that the new implementation is two times faster than the former one. In addition, new experiments were presented comparing the NOP applications to equivalent implementations based on Oriented-Object Paradigm (OOP) in C++. The NOP applications presented, in some cases, better performance than the OOP applications.

Keywords— Notification-Oriented Paradigm, NOP Framework 2.0, Performance comparison.

I. INTRODUÇÃO

O CHAMADO Paradigma Orientado a Notificações (PON) ou, em inglês, *Notification-Oriented Paradigm* (NOP), é uma nova técnica orientada a regras e notificações para o desenvolvimento de software. O PON apresenta uma abordagem baseada em regras na qual estrutura e modelo de execução são baseados em entidades simples que colaboram por meio de notificações diretas entre elas para fins de inferência lógico-causal [1]. Isto permite realizar um processo de inferência evitando redundâncias estruturais (i.e. repetição de código) e redundâncias temporais (i.e. reavaliação desnecessária de código) implícitas [1][2]. Devido a essas propriedades, o PON objetiva ser uma alternativa para melhorar o desempenho na execução de programas em relação a outras técnicas, além de proporcionar características como a

modularidade, a estruturação e o desacoplamento de código [3]. Ademais, por se orientar a regras, o PON tende a facilitar a codificação.

Com a intenção de validar estas características do PON, uma implementação chamada de Framework 1.0 foi desenvolvida em C++. Entretanto, esta implementação demonstrou parcialmente as características do paradigma, mas não precisamente como esperado à luz de sua teoria [3][4]. Em especial, no tocante a desempenho, o Framework 1.0 apresenta problema devido ao uso de estruturas de dados padrões do C++ que são consideravelmente custosas em termos computacionais para o PON.

Neste contexto, uma nova implementação do framework PON foi elaborada e denominada Framework 2.0. Este artigo apresenta esta nova implementação, salientando o que se obteve de avanços a luz das características do PON. Ademais, este artigo apresenta também melhoras no processo de notificação por meio da definição de padrões de notificação.

As próximas seções deste artigo estão organizadas da seguinte maneira. A Seção II contextualiza os conceitos dos principais paradigmas de programação tradicionais. A Seção III, por sua vez, apresenta os fundamentos do PON. A Seção IV, particularmente, apresenta a estrutura otimizada do Framework 2.0, reapresentando seus principais conceitos sob o viés de padrões de projeto. Ainda, nesta seção são apresentadas as novidades no estado da arte e da técnica propostas e implementadas nessa nova versão do framework. A Seção V apresenta um conjunto de aplicações variadas, comparando seus tempos de execução nas duas versões do framework e também com implementações em C++ tradicionais. Por fim, a Seção VI apresenta as considerações finais e perspectivas de trabalhos futuros.

II. CONTEXTUALIZAÇÃO

A capacidade de inferência é essencial para qualquer sistema de decisão, uma vez que permite obter conclusões a partir de estados de entidades ou fatos relacionados, visando ações a serem executadas e determinando, assim, o comportamento do sistema. Os resultados do processo de inferência são conclusões tiradas a partir do relacionamento entre fatos e estruturas de decisão que compõem o sistema. Entre outros, estas estruturas de decisão podem ser organizadas na forma de regras [5]. Tais regras podem ser matemáticas e/ou regras causais, dependendo da natureza do sistema. Um sistema de software, por exemplo, geralmente é constituído principalmente por regras lógico-causais.

A capacidade de inferência lógico-causal em software é normalmente englobada em um dado Sistema de Inferência que pode ser algo explícito ou implícito [6]. Neste sentido,

A. F. Ronszcka, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, ronszcka@alunos.utfpr.edu.br

G. Z. Valença, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, valenca@alunos.utfpr.edu.br

R. R. Linhares, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, linhares@utfpr.edu.br

J. A. Fabro, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, fabro@utfpr.edu.br

P. C. Stadzisz, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, stadzisz@utfpr.edu.br

J. M. Simão, Universidade Tecnológica Federal do Paraná (UTFPR), Curitiba, Paraná, Brasil, jeansimao@utfpr.edu.br

softwares desenvolvidos em linguagens de programação tradicionais no âmbito da Programação Procedimental e a Programação Orientada a Objetos (POO) do Paradigma Imperativo (PI), como C/C++, Java e C#, apresentam em sua essência um sistema de inferência implícito. Esta inferência é baseada em estruturas de decisão e de repetição sobre variáveis e demais elementos passivos, o que leva a uma execução orientada a pesquisas e comparações realizadas com base nos valores de tais variáveis e demais elementos [7].

A programação baseada nos princípios do PI frequentemente provoca acoplamentos em seus elementos de decisão (e.g. aninhamentos de testes condicionais dentro de laços condicionais), tendendo a produzir redundâncias estruturais (repetição de código) e redundâncias temporais (reavaliação desnecessária de código). Isto impacta negativamente no tempo de execução de programas e, principalmente, dificulta a modularização e, portanto, o reuso das partes do software [1][7][8].

Por sua vez, um exemplo de sistema de inferência explícito são os chamados Motores de Inferência. Este tipo de inferência se enquadra no Paradigma Declarativo (PD), o qual se baseia em um estilo de programação orientado a regras. No PD, elementos de decisão (i.e. base de regras lógico-causais) e elementos factuais-execucionais (e.g. base de fatos na forma de *frames*-objetos) são definidos separadamente e relacionados justamente por um sistema de inferência explícito [9][10].

Em suma, o Motor de Inferência é o mecanismo responsável pela execução de um programa orientado a regras, como os Sistemas Baseados em Regras (SBR). Um programa regido pelos princípios do PD geralmente suporta um nível maior de abstração lógico-causal e facilita a programação, quando comparado a um programa implementado em uma linguagem do PI [9][10].

Apesar do PD facilitar em algo a programação, normalmente os algoritmos de inferência utilizados no casamento ou *matching* dos fatos com as regras também apresentam algumas redundâncias estruturais e temporais [1][7]. Isso ocorre, principalmente, pelo modo como a inferência é realizada. Basicamente, algoritmos baseados em busca percorrem um conjunto de elementos da base de fatos, verificando o estado de cada elemento de forma a confrontá-los com regras que podem ou não ser aprovadas em função disto. Neste âmbito, frequentemente, tais elementos são reavaliados desnecessariamente, causando assim redundâncias durante a execução do programa [1].

Algumas soluções do PD, entretanto, reduzem a ocorrência de redundâncias de modo a otimizar a execução, tais como os SBR baseados no difundido algoritmo Rete. No entanto, tais soluções não resolvem completamente o problema e geralmente são compostas por estruturas de dados computacionalmente custosas [11][12][13]. Ainda, estas soluções baseadas em inferência por busca em base de conhecimento passiva (compostas por fatos e regras), mantêm os elementos da base de fatos e os elementos da base de regras fortemente interdependentes, o que pode ser entendido como uma forma de acoplamento. Este tipo de solução orientada a

busca impõem um processo de inferência monolítico. Isto dificulta a paralelização ou distribuição do processamento [1].

Em conjunto, tais limitações frequentemente comprometem o desempenho pleno do sistema de inferência, tanto em sistemas monoprocessados quanto distribuídos. Assim, existem motivações para buscas de alternativas de programação eficientes, com o objetivo de diminuir as deficiências salientadas, mas mantendo ainda as vantagens de SBR [1].

Neste âmbito, uma alternativa a SBR é o Paradigma Orientado a Notificações (PON), o qual foi concebido a partir de uma teoria de Controle e Inferência Orientado a Notificação [14]. O PON se propõe a eliminar algumas das deficiências dos paradigmas tradicionais (particularmente PI e PD) em relação a avaliações causais desnecessárias e acopladas. Isto se dá evitando o processo de inferência monolítico baseado em pesquisas. No PON, há um mecanismo alternativo baseado no relacionamento de entidades computacionais notificantes [1].

III. PARADIGMA ORIENTADO A NOTIFICAÇÕES

O PON encontra inspirações na POO, tais como a flexibilidade algorítmica e a abstração em forma de classes/objetos de elementos factuais. O PON também aproveita conceitos próprios de SBR, como facilidade de programação em alto nível e a representação do conhecimento lógico-causal em regras. Desta forma, o PON provê a possibilidade de uso (de parte) de ambos os estilos de programação em seu modelo, ao mesmo tempo que o evolui (até certo ponto) no tocante ao processo de inferência ou cálculo lógico-causal [15].

O PON apresenta em sua essência a solução para alguns dos problemas dos paradigmas de programação tradicionais como repetição de expressões lógicas e reavaliações desnecessárias destas (i.e. redundâncias estruturais e temporais), bem como o acoplamento forte de entidades no tocante às avaliações ou cálculo lógico-causal. Em linhas gerais, o PON apresenta outra maneira de realizar estas avaliações ou inferências por meio de entidades computacionais de pequeno porte, reativas e desacopladas que colaboram por meio de notificações pontuais, criadas a partir do ‘conhecimento’ de regras [15].

Basicamente, a execução de um programa em PON ocorre de forma reativa, onde um elemento da base de fatos passa a ter autonomia e, ao ter seu valor alterado, notifica precisamente apenas as regras que fazem referência a este. Em um programa elaborado de acordo com o PON, não apenas os elementos da base de fatos, mas inclusive outros componentes, como as partes de uma regra causal, apresentam um comportamento autônomo e reativo.

Todo o processo de execução é separado em partes mínimas, de modo a evitar a existência de elementos repetidos (redundância estrutural), formando assim uma rede de notificações sob demanda (evitando a redundância temporal) entre todas as entidades que compõem a inferência de um dado programa.

A. Inferência Orientada a Notificações

O PON possui instâncias que tratam dos elementos da base de fatos, que são genericamente modelados pela classe FBE (acrônimo de *Fact Base Element*), conforme a Fig. 1. Cada FBE contém atributos na forma de instâncias da classe *Attribute* e seus serviços na forma de instâncias da classe *Method*. As instâncias FBE, via seus *Attributes* e *Methods*, são passíveis de correlação causal por meio de *Rules*, as quais se constituem em elementos fundamentais do PON [1].

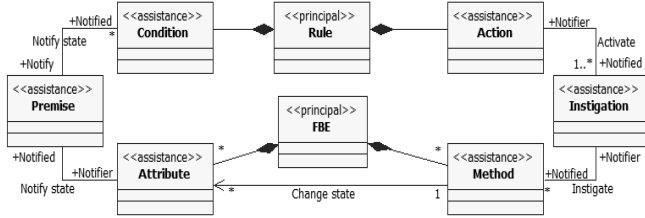


Figura 1. Entidades do PON e seus relacionamentos por notificação.

A Fig. 2 apresenta um exemplo de *Rule*, na forma de uma regra causal. Embora seja possível representá-la desta forma, cada *Rule* é efetivamente uma entidade computacional composta por outras entidades, conforme Fig. 1, que podem ser vislumbradas do ponto de vista de instâncias/classes. Por exemplo, a *Rule* apresentada é composta por uma instância *Condition* e uma instância *Action*. A *Condition* trata da decisão da *Rule*, enquanto a *Action* trata da execução das ações da *Rule*. Portanto, *Condition* e *Action* trabalham juntas para realizar o conhecimento lógico e causal da *Rule* [15].

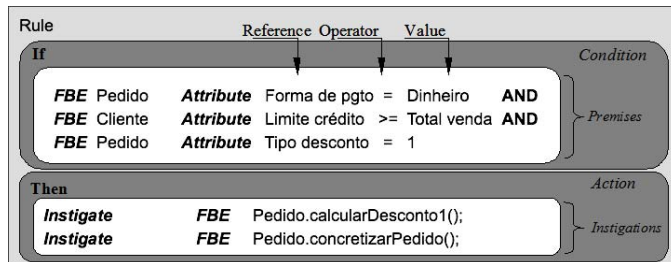


Figura 2. Exemplo de uma *Rule* em PON.

Esta *Rule* apresentada na Fig. 2 faria parte de um sistema de pedidos de venda. A *Condition* desta *Rule* lida com a decisão de validar se as *Premises* em questão aprovam determinadas características do pedido considerado: a) A forma de pagamento é em dinheiro? b) O limite de crédito do cliente é compatível com o valor total da venda? c) O tipo de desconto aplicado é de 5% (tipo 1)? Assim, é possível concluir que os estados dos *Attributes* dos *FBEs* compõem os fatos a serem avaliados pelas *Premises*.

Na verdade, cada *Premise* avalia o estado de um ou mesmo dois *Attributes* de *FBE*. Para cada mudança de estado de um *Attribute* do *FBE*, este envia notificações somente e precisamente para as *Premises* que dele dependem. Estas, por sua vez, efetuam a sua correspondente avaliação lógica. Similarmente, a partir da mudança de estado das *Premises*, estas enviam notificações somente e precisamente para as

Conditions que dela dependem, as quais efetuam a sua correspondente avaliação causal. Isto tudo se dá por meio de uma cadeia de notificações entre instâncias notificantes e minimamente acopladas, o que se constitui no fundamento do PON conforme modelado na Fig. 1 e esboçado na Fig. 3.

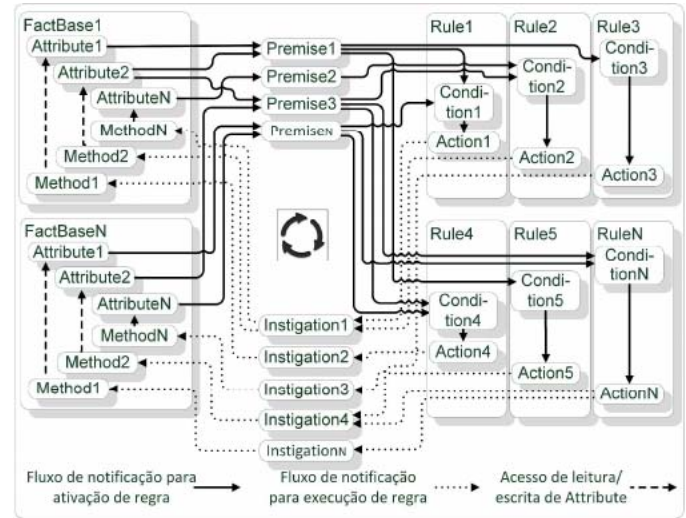


Figura 3. Inferência por notificações.

Ainda, uma *Action* também é uma entidade que se conecta a entidades de outro tipo, as *Instigations*. Neste exemplo, a *Action* contém duas *Instigations* para: a) calcular e conceder o desconto do tipo 1 para este pedido; e b) finalizar e concretizar o pedido. Efetivamente, o que uma *Instigation* faz é instigar um ou mais métodos responsáveis por realizar serviços de um *FBE*.

Cada método de um *FBE* é também tratado por uma entidade chamada de *Method*. A execução de um *Method* potencialmente muda o estado de um ou mais *Attributes*, iniciando assim outras cadeias de notificações [1].

Em suma, cada *Attribute* notifica apenas as *Premises* relevantes sobre seus estados somente quando se fizer efetivamente necessário. Cada *Premise*, por sua vez, notifica apenas as *Conditions* relevantes a partir de mudanças em seus estados usando o mesmo princípio. Baseado nestes estados notificados é que a *Condition* pode ser aprovada ou não. Se a *Condition* é aprovada, a respectiva *Rule* pode ser ativada executando sua *Action*. Em tempo, o conhecimento de quem se deve notificar se dá na composição das *Rules*, em um dado ambiente de desenvolvimento para o PON [15].

Com isto considerado, é possível observar que a essência da computação no PON está organizada, desacoplada e distribuída entre entidades autônomas e reativas que colaboram por meio de notificações pontuais. Este arranjo forma o mecanismo de notificações, o qual determina o fluxo de execução das aplicações. Por meio deste mecanismo, as responsabilidades de um programa são divididas entre as entidades do modelo, o que proporciona uma execução otimizada e ‘desacoplada’ (i.e. minimamente acoplada), útil para o aproveitamento correto de monoprocessamento, bem como para o processamento paralelo/distribuído [14][15].

A luz das técnicas existentes, pode-se dizer que o PON

permite desenvolvimento orientado a regras em alto nível, ao mesmo tempo em que utiliza elementos de programação reativa e orientada a eventos para alcançar um arranjo particular de inferência. Neste arranjo, os elementos factuais (usualmente variáveis, atributos-objetos, frame-slots e afins) notificam elementos lógico-causais (regras, expressões sentença e afins) para alcançar uma nova forma de inferência. No PON, entretanto, tudo ocorre por notificações (similares a eventos, interrupções e afins) nos construtos mais elementares de programação, o que se constitui na sua contribuição ao estado da arte enquanto nova técnica de programação.

B. Implementação do PON

Em termos de estado da técnica, o PON tem sido implementado por meio de um conjunto particular de classes e instâncias da POO (mais precisamente na linguagem de programação C++). Tais classes e instâncias orientadas a notificações permitem realizar o comportamento de cada uma das entidades que compõem o processo de inferência do PON. Estes elementos, em conjunto com outros mecanismos que viabilizam o tratamento de conflitos de execução, formam um framework de desenvolvimento de programas para este paradigma. Esta abordagem é bastante comum, sendo que, geralmente, os paradigmas emergentes têm sua essência implementada em outros paradigmas até que seus fundamentos sejam estabelecidos e se justifique a existência de um modelo próprio (e.g. linguagem e compilador).

Isto dito, desde a concepção prototipal do framework do PON [1][14], até uma concepção dita primária ou original (versão 1.0) [15], possibilitou-se a criação de aplicações regidas sob os princípios do PON. O Framework 1.0 apresenta uma estrutura mais eficiente em comparação à versão prototipal. Enquanto a versão prototipal apresentava um esboço das principais entidades do PON, a versão 1.0 proporcionava um controle de criação de entidades razoavelmente desacopladas e mecanismos para a resolução de conflito entre regras.

A estrutura geral do Framework 1.0 contempla a criação de programas dinâmicos por meio de classes genéricas, conforme apresentado na Fig. 1. A concepção de um programa em PON se dá inicialmente por meio da instanciação de objetos da classe FBE, nas quais o desenvolvedor define os *Attributes* e *Methods*, que as compõem. A concepção de um programa em PON segue com a instanciação de objetos da classe *Rule* que criam todas as amarrações internas com as demais entidades (i.e. *Condition*, *Premise*, *Action*, *Instigation*, *Attributes* e *Methods*).

No tocante ao código, cada uma das entidades apresenta em sua estrutura interna uma lista encadeada, baseada em componentes da *Standard Template Library (STL)* do C++, que contém a referência de todas as entidades que apresentam interesses em suas mudanças de estado. Nesse caso, ao acontecer mudança de um estado, a entidade notifica todas as entidades em sua lista. Esse mecanismo de notificações de uma entidade para a outra, forma o fluxo de execução de um programa PON, o qual não é dependente de um laço principal que regeria o fluxo de execução [16].

Apesar do Framework 1.0 facilitar a criação de programas baseados no PON, a sua estrutura apresenta algumas deficiências que precisavam ser corrigidas e aprimoradas. A remodelagem deste framework levou em conta vários princípios e padrões de projeto e de implementação, que tornaram sua estrutura mais coesa, desacoplada e otimizada [17].

IV. FRAMEWORK PON 2.0

De maneira geral, as deficiências relacionadas ao desempenho das aplicações desenvolvidas com o Framework 1.0 motivaram refatorações em seu código, em especial no processo de notificações, de maneira a possibilitar a utilização de diferentes estruturas de dados e seus respectivos iteradores.

Além das questões relacionadas ao desempenho de execução, as versões precedentes do framework apresentavam problemas em suas estruturas, que não contribuíam para a evolução mais acentuada do uso e da implementação do PON. Tais deficiências dificultavam a proposta de melhorias e novos recursos, principalmente por terem sido desenvolvidas de forma um tanto acoplada e pouco coesa, o que se contradiz com a própria teoria do PON.

Neste âmbito, esta seção apresenta as otimizações propostas para esta nova versão do framework. Esta seção também reapresenta a estrutura do PON sob uma nova perspectiva, sob o viés de padrões. Isto tem o intuito de facilitar seu entendimento visando a universalização dessa forma de expressão na computação.

De maneira geral, as notificações entre entidades PON ocorrem quando o estado de uma destas é alterado. Geralmente, quando o estado de uma entidade é modificado, ela precisa notificar uma ou mais entidades relacionadas sobre tal mudança. A implementação do conjunto de entidades interessadas a serem notificadas foi baseada no uso de uma lista encadeada. Entretanto, a implementação adotada no Framework 1.0 foi o tradicional contêiner *list* da *STL* do C++, o qual apresenta recursos como navegações bidirecionais e/ou operações de ordenação rebuscadas, que impactam consideravelmente no desempenho de execução das notificações. Tais recursos avançados não são importantes no processo de notificação pontual do PON [16].

Neste âmbito, foram implementadas outras estruturas de dados no Framework 2.0, possibilitando que as notificações entre entidades PON se adequassem às particularidades de diversos domínios de aplicações distintos. Dentre as estruturas de dados disponíveis no atual Framework 2.0, além da possibilidade de poder continuar usando a *list* da *STL*, há agora três novas estruturas de dados implementadas especificamente para atender as necessidades do framework, nomeadamente PONLIST, PONVECTOR e PONGHASH [16]. Cada nova estrutura implementada apresenta vantagens e desvantagens que variam de acordo com as particularidades das aplicações.

A estrutura de dados denominada PONLIST é uma implementação purista de uma lista encadeada unidirecional. Tal implementação valoriza a simplicidade de iteração sobre as entidades do PON. Esta implementação mais simplificada

otimiza o processo em relação ao realizado pela *list* da *STL*.

A utilização da estrutura denominada *PONVECTOR*, por sua vez, tem por objetivo realizar uma melhor utilização da organização dos dados na memória. Desta forma, o armazenamento dos elementos sobre a estrutura *PONVECTOR* é realizado de forma sequencial, por meio de um vetor de elementos. Neste sentido, a iteração dos elementos *PON* é otimizada por meio da utilização de aritmética de ponteiros.

Por sua vez, a estrutura *PONHASH* foi implementada de maneira a realizar notificações pontuais, focando as entidades que desejem receber notificações somente para um estado (valor) específico da entidade notificante. Para isso, é necessário realizar de antemão o cálculo da função *hash* para direcionar as notificações apenas para as entidades interessadas. Ademais, as entidades que haviam sido notificadas por estarem interessadas no estado anterior são igualmente notificadas para terem seus estados lógicos reavaliados. A Fig. 4 ilustra esse processo de notificações.

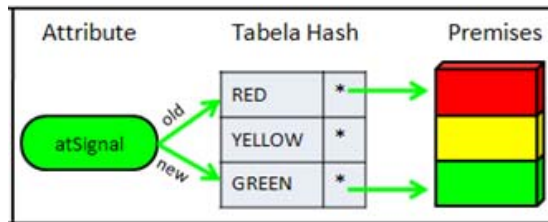


Figura 4. Notificações baseadas na estrutura *PONHASH* [15].

Na Fig. 4, o *Attribute atSignal* ao receber um novo estado (*Green*) só irá notificar as *Premises* interessadas no antigo estado desse *Attribute* (*Red*) e as *Premises* interessadas no estado atual (*Green*), desconsiderando todos os demais estados que não teriam nenhum efeito a partir dessa notificação (*Yellow*, neste exemplo).

Tais estruturas representam o cerne do processo de notificações entre entidades *PON*. Esse mecanismo afeta diretamente o desempenho dos programas desenvolvidos com o framework, uma vez que notificações desnecessárias podem ser suprimidas em determinados casos.

Ademais, de maneira geral, o Framework 2.0 foi reconstruído baseado em um conjunto de padrões de implementação e de projeto. Essa reestruturação focou, principalmente, em reduzir o acoplamento na estrutura do framework, mais precisamente no processo de notificações entre as entidades elementares do *PON*. Esta reestruturação tornou o framework mais flexível para seu uso geral, bem como deixou sua estrutura mais genérica de modo a facilitar a extensão e aprimoramentos futuros [17].

A. Padrão *Observer*

Diferentemente da maneira passiva de inferência encontrada na maioria dos paradigmas tradicionais, o *PON* “evolui” as entidades, dotando-as com características reativas, o que permite que essas reajam apenas a mudanças em seus estados lógicos via notificações pontuais, evitando avaliações redundantes. No lugar de pesquisa e iterações sobre entidades

passivas (e.g. comandos e variáveis) ou semipassivas (e.g. módulos com alguma reatividade), ocorrem as notificações entre as entidades colaboradoras pertinentes [17].

Neste sentido, cada relação baseada em notificações existente no *PON* pode ser entendida como um caso particular e pontual de aplicação do Padrão *Observer* para fins de cálculo lógico-causal. A utilização do padrão *Observer* em aplicações de naturezas distintas é vasta, geralmente aplicada na composição de entidades maiores (e.g. sistemas, agentes, módulos). Entretanto, no *PON* e seu framework, o conceito é utilizado em um nível mais “baixo” ou “atômico” aos comumente aplicados.

Na verdade, o conceito do padrão *Observer* é extrapolado no *PON*, uma vez que sua aplicação está presente na essência da inferência ou cálculo lógico-causal desse paradigma. Mais precisamente, o padrão *Observer* é extrapolado uma vez que está presente na aprovação ou desaprovação de cada entidade lógico-causal (e.g. “se-então”) em função dos valores ou estados de cada entidade factual (e.g. “atributo ou variável”) pertinente. Neste âmbito, o modelo conceitual do padrão *Observer* é aplicado ao modelo do *PON*, conforme ilustrado no diagrama de classes exposto na Fig. 5, sendo que esse modelo se constitui no núcleo do Framework 2.0.

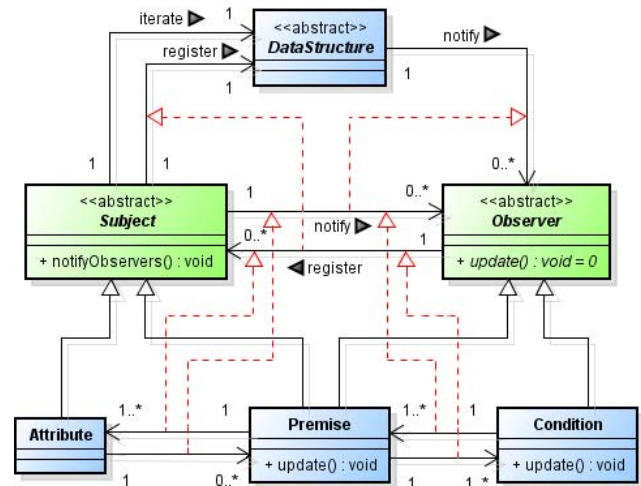


Figura 5. Padrão *Observer* aplicado no processo de notificações [17].

Conforme ilustra a Fig. 5, a estrutura do padrão *Observer* é composta por duas classes principais, isto é, a classe *Subject* e a classe *Observer*. Cada instância da classe *Subject* é responsável pelas notificações pontuais às instâncias interessadas em seu estado (i.e. observadores). Por sua vez, cada instância da classe *Observer* é responsável pela execução de suas responsabilidades a cada notificação recebida a partir da instância da classe *Subject*. Ademais, os *Observers* são acomodados em estruturas de dados (i.e. *DataStructure*), as quais organizam tais entidades de maneira a facilitar as notificações pontuais por parte da classe *Subject*.

De maneira geral, no Framework 2.0, a entidade *Attribute* apresenta o comportamento de notificadora, enquanto a entidade *Premise* observa toda e qualquer mudança que ocorra em um *Attribute* relacionado. Esse mesmo comportamento ocorre em relação às *Premises* e às *Conditions*, uma vez que

as *Conditions* se interessam pelas mudanças de estados das *Premises* relacionadas.

B. Padrão Observer

De maneira geral, em termos práticos, cada entidade do PON depende de um meio para referenciar as entidades a serem notificadas. Na programação, cada entidade do PON depende, portanto, de uma dada estrutura de dados (supostamente otimizada e apropriada) para armazenar tais referências. Ainda, a iteração de tal estrutura de dados deveria se dar sem maiores conhecimentos de sua natureza interna de processamento, generalizando assim a solução para o tratamento de notificações em cada entidade pontual do PON. Sendo assim, o padrão de projeto *Iterator* foi aplicado no âmbito do percurso das estruturas de dados existentes. Para isso, o diagrama de classes exposto na Fig. 6 apresenta o modelo conceitual do padrão *Iterator* aplicado ao Framework 2.0 do PON [17].

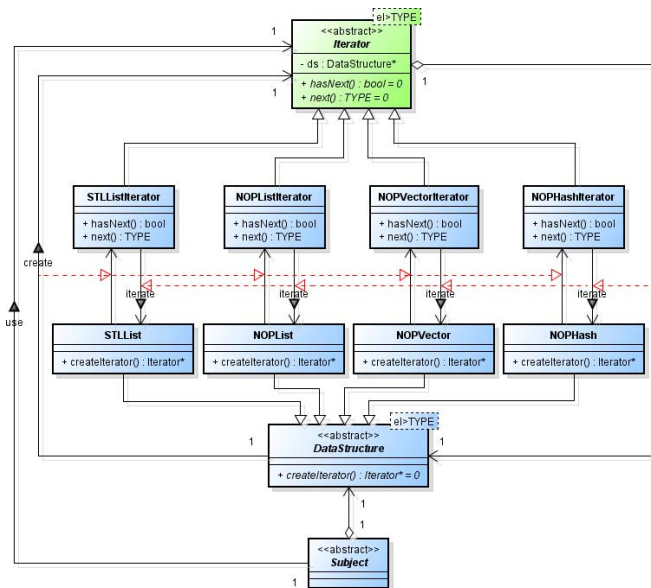


Figura 6. Padrão *Iterator* aplicado no processo de notificações pontuais [17].

Conforme ilustra o diagrama de classes da Fig. 6, a classe principal *Iterator* é responsável por comportar uma estrutura de dados e fornecer métodos para acessar sequencialmente as referências das entidades armazenadas em sua estrutura. Ainda, a classe *DataStructure* é responsável por criar o iterador, passando uma instância dela mesma como referência para ele.

Em tempo, no Framework 2.0, as diversas estruturas de dados criadas apresentam particularidades distintas na forma que armazenam e percorrem as referências para as entidades armazenadas. Entretanto, a utilização desse padrão possibilita que todas as estruturas de dados sigam um modelo padronizado, facilitando a sua utilização (polimorficamente) na classe cliente. Mais precisamente, os métodos *hasNext* e *next* da classe *Iterator* são definidos como virtuais puros (i.e. abstratos), enquanto as classes concretas obrigatoriamente implementam as particularidades desses métodos de acordo com a estrutura de dados pertinente.

C. Padrão Observer

A necessidade de criar diferentes estruturas de dados para ganhos de desempenho suscitou problemas relacionados à dificuldade de composição e instanciação de entidades PON. Tal problema é solucionado ao aplicar o padrão de projeto *Abstract Factory* na criação de tais entidades, uma vez que esse fornece uma interface abstrata e única para tal. A partir dessa interface, cada família de entidades (compostas por diferentes estruturas de dados) apresenta uma fábrica específica para a criação de tais entidades polimorficamente. De maneira a explicitar essa explicação, a Fig. 7 ilustra a aplicação desse padrão na estrutura do Framework 2.0 [17].

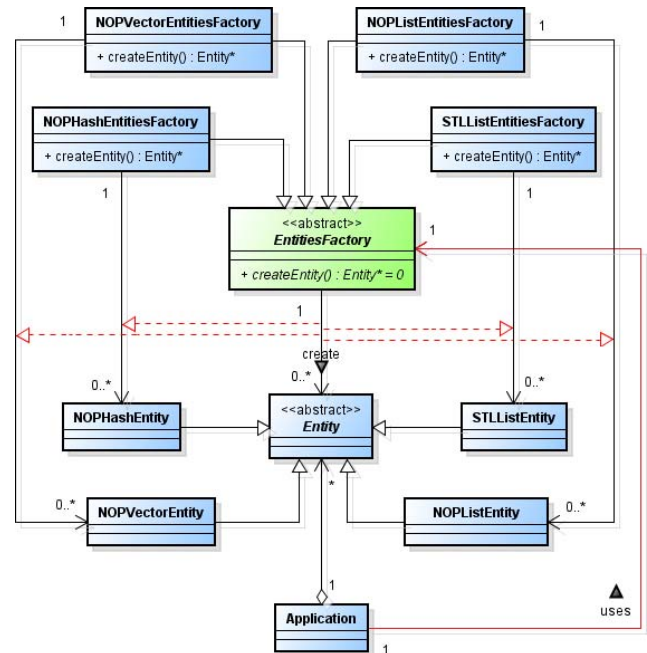


Figura 7. Padrão *Abstract Factory* aplicado à criação de entidades PON [17].

Conforme demonstra a Fig. 7, a classe principal desse padrão é a fábrica abstrata (i.e. *EntitiesFactory*). Essa classe é responsável pela definição de todos os métodos abstratos responsáveis pela criação de entidades PON. O diagrama, em particular, abstrai a implementação real com a representação da classe abstrata *Entity*, a qual simboliza as entidades PON (e.g. *Attribute*, *Premise*, *Condition*, *Action*). Isto é, cada entidade PON básica, derivada de *Entity*, possui sua própria construção particular para cada uma das estruturas de dados presentes no Framework 2.0.

Ademais, o diagrama da Fig. 7 ilustra a presença de outras fábricas, ditas concretas, que têm por finalidade implementar os métodos herdados da fábrica abstrata. Tais métodos têm por finalidade a criação de entidades PON concretas, as quais implementam internamente suas respectivas particularidades para cada estrutura de dados existente.

Particularmente, o padrão *Abstract Factory* flexibiliza a criação de entidades PON compostas por diferentes estruturas de dados, ao possibilitar a criação dessas com o uso de uma interface única (i.e. fábrica abstrata). Outrossim, tal padrão viabiliza a criação de novas famílias de entidades PON, compostas por eventuais novas estruturas de dados que

possam vir a ser implementadas em trabalhos futuros.

D. Padrão Observer

A criação facilitada de entidades PON proporcionada pelo padrão de projeto *Abstract Factory* exige das classes que o utilizam apenas que possuam uma referência (ponteiro) para uma fábrica concreta. Desta forma, os métodos de criação fornecidos pela fábrica genérica intermediam a criação de tais entidades *polimorficamente* sem a necessidade do conhecimento prévio de como isso é feito internamente pelas classes concretas [17].

Neste âmbito, de modo a facilitar o acesso a tal fábrica, bem como garantir que exista apenas uma única instância de uma fábrica concreta em tempo de execução, o padrão de software *Singleton* foi implementado na estrutura do Framework 2.0. Ainda, o padrão garante que tal fábrica crie entidades PON de maneira uniforme, de modo que todas as entidades criadas sejam formadas pela estrutura de dados definida na configuração inicial do programa. A estrutura do padrão *Singleton* suportando a viabilidade do padrão *Abstract Factory* pode ser visualizada na Fig. 8.

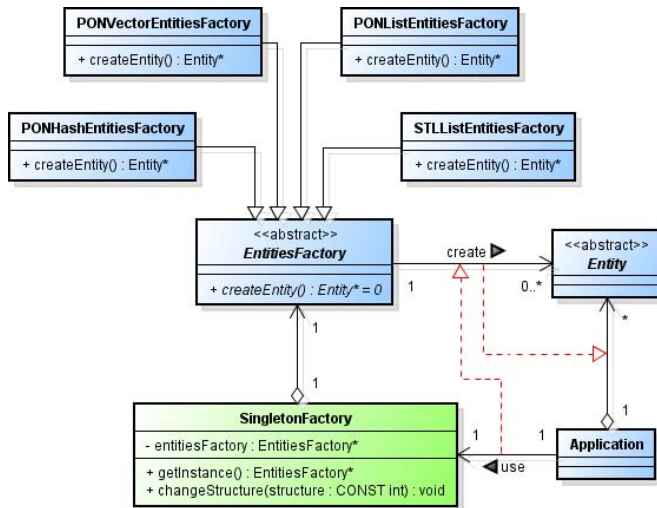


Figura 8. Padrão *Singleton* aplicado no processo de definição de fábricas [17].

Conforme ilustra o diagrama de classes da Fig. 8, a classe centralizadora *SingletonFactory* é responsável por manter uma referência para uma fábrica de entidades PON. Além disso, essa classe, ao seguir o padrão de software *Singleton*, assegura que apenas uma única instância de tal fábrica seja utilizada na criação de entidades PON.

E. Padrão Strategy

Apesar de o padrão *Singleton* garantir a existência de apenas uma instância de uma fábrica concreta de entidades PON, conforme dito anteriormente, há casos particulares em que entidades compostas por outra estrutura de dados poderiam apresentar maior eficiência em termos de consumo de processamento em um dado contexto. Neste sentido, as entidades PON não precisariam necessariamente se adequar a uma estrutura de dados única em todo o sistema, uma vez cada entidade incorpora um mecanismo de notificações interno exclusivo. Esse fato se mostra favorável à composição de

aplicações PON mais eficientes em questões de desempenho, onde cada entidade poderia ser composta pelo mecanismo mais apropriado para cada situação [17].

Deste modo, o padrão de projeto *Strategy* foi implementado na arquitetura do Framework 2.0 de modo a possibilitar a alteração da instância da fábrica em tempo de execução (i.e. por meio do método *changeStructure*), permitindo assim a criação de entidades compostas pela última estrutura configurada no padrão *Singleton*. Outrossim, é importante ressaltar que esse padrão não inviabiliza a presença do padrão *Singleton*, uma vez que esse ainda atuaria para garantir a centralização da criação de entidades PON em uma interface única.

A combinação desses dois padrões em questão, juntamente com o padrão *Abstract Factory*, possibilita que a classe responsável por armazenar uma referência para uma fábrica concreta possa receber referências de outras fábricas em tempo de execução, sem afetar de fato no fluxo de execução do programa. Com isso, entidades PON podem ser criadas deliberadamente sem influenciar no comportamento umas das outras, devido ao acoplamento mínimo proporcionado pelo Framework 2.0.

F. Padrão Observer

O processo de notificações no Framework 2.0 é constituído por todas as entidades que compõem a estrutura do PON definidos em sua teoria (cf. Fig. 1). Entretanto, mesmo teoricamente, há distinções na maneira com que essas entidades se notificam [17].

Por um lado, as entidades responsáveis pelo cálculo lógico-causal apenas notificam as entidades interessadas em mudanças pontuais em seus estados lógicos. Tal mecanismo, conforme explicado anteriormente, foi concebido com a aplicação do padrão *Observer*. Por outro lado, as entidades responsáveis pela execução de uma *Rule*, quando aprovada, seguem um mecanismo de execução levemente diferente, notificando ou simplesmente instigando/comandando as demais entidades colaboradoras pertinentes para que executem determinada ação. A Fig. 9 ilustra o diagrama de classes que explicita a execução de uma *Rule* PON.

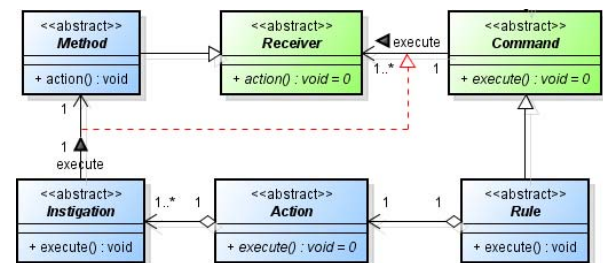


Figura 9. Padrão *Command* aplicado na execução de *Rules/Methods* [17].

Conforme ilustrado no diagrama de classes exposto na Fig. 9, a aplicação do padrão de projeto *Command* no Framework 2.0 apresenta duas classes principais. A classe *Command*, responsável pela ordem de execução de uma ação, tem uma relação indireta com a classe *Receiver*, a qual é responsável pela execução de tal ação.

Ademais, tais classes colaboradoras, cada qual com suas funções específicas, apresentam como uma de suas responsabilidades a delegação de execuções de uma determinada ação para as entidades apropriadas. Neste sentido, a classe *Rule* delegaria a execução para a classe *Action* que, por sua vez, delegaria a execução para a classe *Instigation*, que por fim instigaria a execução da classe *Method*. Como resultado final, cada *Rule (Command)* aprovada executaria a ação de um ou mais *Methods (Receiver)*, possibilitando que a execução desse(s) seja(m) realizada de maneira independente.

V. CASOS DE ESTUDO – APLICAÇÕES PON

Esta seção apresenta duas aplicações de naturezas distintas, propostas para o estudo em questão. Esta seção descreve o escopo de cada aplicação desenvolvida e faz um comparativo em relação a tempos de execução, comparando o desempenho da versão 1.0 do framework com a nova versão proposta neste trabalho, bem como de aplicação equivalente feita em C++.

É importante ressaltar que experimentos preliminares, com base na estrutura de dados PONLIST já foram realizados sob alguns destes programas e tiveram seus resultados publicados nos seguintes trabalhos [18][19][20]. Os resultados mostraram uma melhora significativa em termos de desempenho de execução em relação ao Framework 1.0. As conclusões destes trabalhos impulsionaram a evolução da ferramenta, para a qual foram propostas novas estruturas de dados, bem como as demais otimizações que compõem o Framework 2.0 do PON.

Neste âmbito, as subseções seguintes apresentam os resultados obtidos com os novos experimentos. A subseção *A* apresenta a aplicação Mira ao Alvo, enquanto a subseção *B* apresenta um sistema de controle de vendas.

A. Aplicação Mira ao Alvo

O primeiro programa utilizado nos experimentos é chamado de Mira ao Alvo. Este programa representa um *toy problem* no qual arqueiros devem atirar flechas em maçãs dada uma condição de prontidão [3][15].

O programa Mira ao Alvo explora, principalmente, o fato de existirem redundâncias estruturais (comparar quais arqueiros podem atirar e quais não podem) e redundâncias temporais (comparações repetidas e desnecessárias, até que os arqueiros estejam prontos) em programas baseados em paradigmas predominantes. Ademais, esse programa tem sido utilizado como ambiente para testes nas ferramentas e técnicas desenvolvidas pelo grupo que desenvolve o PON, uma vez que abstrai bem a essência de programas de maior escala, nos quais as redundâncias tendem a estar dispersas na estrutura do programa.

Basicamente, a aplicação consiste em um ambiente no qual as entidades do tipo Mira interagem ativamente com as entidades do tipo Alvo. Mais precisamente, as entidades Mira e as entidades Alvo são representadas, respectivamente, por arqueiros e maçãs, sendo que há uma maçã para cada arqueiro. Em termos de implementação, cada arqueiro e maçã são representados por meio de objetos, os quais interagem de acordo com a validação de suas expressões causais

pertinentes. Estas expressões causais relacionam os atributos/estados e métodos/serviços destes objetos.

Neste contexto, cada arqueiro e cada maçã recebem um identificador numérico, sendo que um arqueiro somente pode flechar uma maçã que apresente o identificador numérico correspondente ao seu. Ainda, os arqueiros também apresentam um atributo que denota o estado de pronto (i.e. status) para agir sobre o cenário (i.e. flechar a respectiva maçã). Outrossim, um terceiro elemento, denominado arma de fogo, tem a função de sinalizar, com o seu disparo, o início de uma iteração, permitindo que os arqueiros interajam com as respectivas maçãs.

Em relação as maçãs, estas também apresentam um atributo que denota o seu estado de pronto (i.e. status). Ainda, estas apresentam um atributo que explicita se a mesma já foi perfurada por uma flecha ou ainda não (i.e. *isCrossed*). Também, as maçãs apresentam um atributo que se refere a sua coloração (i.e. *color*), uma vez que as maçãs podem se apresentar em duas diferentes cores: vermelha ou verde.

Ademais, cada arqueiro somente pode interagir com a sua respectiva maçã após a constatação de quatro premissas: (a) se a cor da maçã que está posicionada diretamente em sua frente é vermelha, (b) se a maçã que está posicionada diretamente em sua frente está pronta para ser atingida, (c) se ela é identificada pelo seu número correspondente e ainda, (d) o estado da arma de fogo é atirar.

Se as quatro condições forem satisfeitas, o arqueiro está liberado para atingir a respectiva maçã com a projeção de sua flecha. Desta forma, para cada par de arqueiros e maçãs deve haver uma expressão causal para comparar os seus estados.

De modo a verificar o impacto de *Rules* passíveis de aprovação, bem como *Rules* que não seriam aprovadas na mudança de um estado de um determinado *Attribute* (i.e. estado da arma de fogo), o experimento apresenta três amostras. Na primeira amostra, apenas 10% das *Rules* possuem a cor da maçã em vermelho; aprovando apenas 10% das *Rules* na mudança de estado da arma de fogo para 'pronta'. Na segunda amostra, por sua vez, 50% das *Rules* são ativas com a mudança do estado da arma de fogo. Por fim, na terceira amostra, todas as *Rules* são ativadas.

A Fig. 10 apresenta os testes de desempenho considerando uma aplicação convencional em C++ (POO C++), o Framework original (FW 1.0) e sua versão otimizada, considerando a utilização pontual de cada uma das novas estruturas de dados implementadas para o Framework 2.0. Os tempos são apresentados em milissegundos. A quantidade de iterações processadas foi de cem mil e o percentual de avaliações causais aprovadas variam em 10%, 50% e 100%.

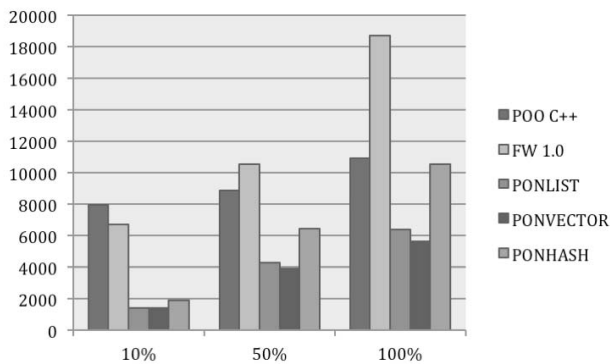


Figura 10. Tempos de execução (em ms) da aplicação Mira ao Alvo.

Conforme observado na Fig. 10, a utilização da estrutura de dados PONVECTOR, pontualmente aplicada em todas as notificações, apresentou em média um tempo de processamento cerca de 3 (três) vezes menor em relação a versão original do Framework 1.0. Ademais, a estrutura PONLIST apresentou tempos próximos aos da estrutura PONVECTOR, com uma pequena diferença que é justificada pela forma com que os ponteiros são armazenados sequencialmente e, principalmente, pela forma com que o percorrimto do vetor é realizado. Ainda, neste cenário, a estrutura PONHASH apresentou um desempenho um pouco inferior comparado às demais estruturas, devido à quantidade de elementos distintos a serem notificados ser relativamente pequena, o que não justifica a sua aplicação nesse cenário.

B. Sistema de Controle de Vendas

Esta subseção apresenta um segundo caso de estudo que consiste na implementação de um sistema de controle de vendas (pedidos de venda) tradicional. Esse programa seria uma tradicional aplicação do estilo *CRUD* (acrônimo de *Create, Retrieve, Update e Delete*) que permite criar, recuperar, atualizar e eliminar dados [19]. Em suma, o escopo da aplicação pode ser vislumbrado no diagrama de atividades apresentado na Fig. 11.

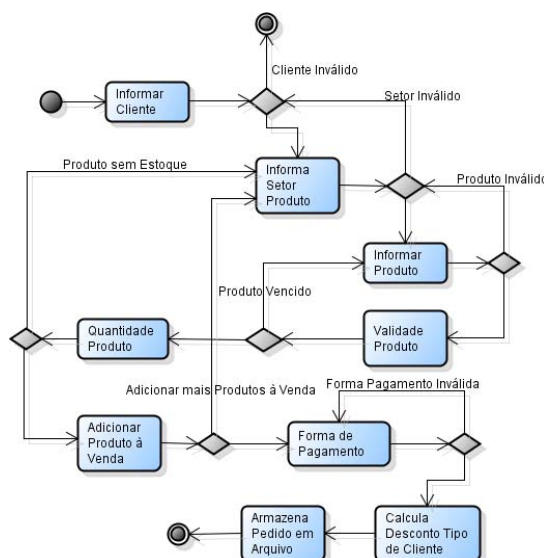


Figura 11. Diagrama de Atividades – Sistema de Controle de Vendas [17].

Conforme ilustrado na Fig. 11, o processo de venda se inicia com o informe do cliente que realizará o pedido. Uma vez escolhido e aprovado a venda para determinado cliente, devem ser informados os produtos que irão compor o pedido. O sistema possui validações quanto à existência de produtos e clientes. Ademais, o estoque de tais produtos é verificado. Se o produto escolhido para venda pertencer ao setor de perecíveis, a sua data de validade é verificada. Na ocorrência de produtos vencidos, a venda não será permitida.

Depois de realizado o ciclo de informe de produtos, a venda poderá ser finalizada após a inserção da forma de pagamento, desconto e armazenamento do pedido. Na implementação desse sistema, existem apenas duas formas de pagamento possíveis, à Vista ou à Prazo. O cliente, em seu cadastro, possui uma informação sobre seu limite de crédito. Caso a forma de pagamento escolhida tenha sido à Prazo, o sistema verifica se o cliente tem permissão para efetuar a compra, confrontando o valor total do pedido com seu limite de crédito. Ainda, no cadastro do cliente há uma informação que lhe concede um tipo de classificação. Tal classificação é utilizada para a concessão de descontos especiais durante a finalização da venda. Para tanto, existe um total de 20 classes de clientes que dispõem de descontos que variam de uma faixa de 5% a 95%. Neste caso, para cada tipo de desconto será criada uma *Rule* correspondente. Desta forma, após a satisfação das *Premises*, a *Rule* em questão concederia o desconto do pedido de vendas para o cliente e, por fim, finalizaria sua venda.

Sendo assim, os pedidos seguem um padrão, possuindo a mesma quantidade de itens para cada produto e desconto. Os experimentos neste cenário levaram em conta a criação de 100 (cem), 1.000 (um mil) e 10.000 (dez mil) pedidos para os casos de teste, sendo realizada a média de 100 repetições/iterações do mesmo experimento para garantir a qualidade dos dados. Os tempos de processamento são apresentados em milissegundos. Os resultados deste experimento são expostos na Fig. 12.

Pedidos	POO C++	FW 1.0	LIST	VECTOR	HASH
100	437	589	325	269	1290
1.000	5766	7589	4269	3533	15890
10.000	60183	69875	53259	50489	114567

Figura 12. Tempos de execução (em ms) do sistema de controle de vendas.

Neste cenário, é possível observar que as estrutura de dados PONLIST e PONVECTOR apresentaram uma redução média de processamento de cerca de 50% em relação ao Framework 1.0. Além disso, neste experimento foram feitas comparações com uma implementação similar deste mesmo sistema seguindo os princípios do POO. Neste experimento, o Framework 1.0 apresentou uma eficiência inferior à implementação em POO. Após as devidas otimizações e melhorias na estrutura do Framework 2.0, foi possível observar uma melhora significativa em termos de desempenho, na qual a estrutura PONVECTOR apresentou uma redução média de processamento de cerca de 30% em relação à implementação no POO.

Por outro lado, a estrutura PONHASH apresentou resultados insatisfatórios neste experimento. O uso desta estrutura é bastante peculiar e não é adequado para todos os casos de notificações, principalmente as notificações realizadas entre entidades com poucas dependências. Neste âmbito, uma das melhorias apresentadas no Framework 2.0 é o desacoplamento das entidades e a forma pela qual estas realizam suas notificações pontuais. Este desacoplamento é apresentado principalmente pela implementação do padrão de projeto *Abstract Factory*. A separação das entidades em modelos genéricos e concretos, nos quais cada qual pode atuar sobre uma estrutura de dados específica, possibilita a criação de programas com entidades compostas por diferentes estruturas de dados.

Neste âmbito, um cenário ideal é constatado na presença de *Attributes* com uma frequência alta de mudança de estados, vinculados a um número relativamente grande de *Premises*. Neste caso, a estrutura de dados denominada PONHASH será sempre uma forte candidata a ser utilizada para comportar esse tipo de entidade.

Desta forma, um novo cenário foi proposto para o sistema de controle de vendas. Neste cenário foram utilizadas estruturas mistas, nas quais o *Attribute* “*atTypeDiscount*” referente aos descontos concedidos para os itens de um pedido, foi definido para atuar sobre a estrutura PONHASH, enquanto as demais entidades do sistema permaneceram sobre a estrutura PONVECTOR.

A Fig. 13 apresenta os resultados obtidos após a utilização de estruturas únicas (PONLIST, PONVECTOR, PONHASH) em comparação com a utilização de estruturas mistas, no caso, a utilização da estrutura PONVECTOR em conjunto com o uso pontual da estrutura PONHASH.

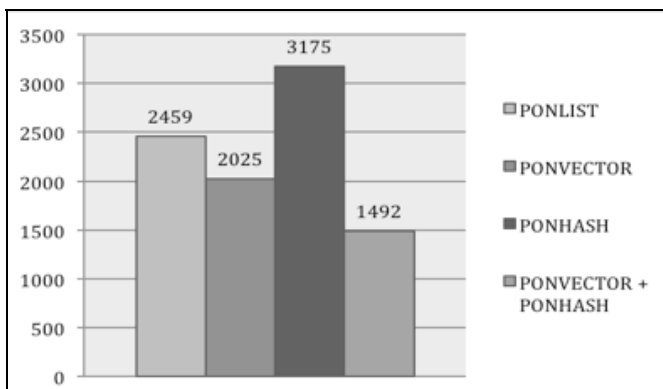


Figura 13. Comparações entre implementações com estruturas PON.

Conforme apresentado na Fig. 13, a utilização mista da estrutura PONVECTOR para as entidades padrão e a estrutura PONHASH para o caso particular de notificação de um *Attribute* ‘pesado’, apresentou um ganho de desempenho médio de cerca de 35% em relação a utilização exclusiva da estrutura PONVECTOR para todas as entidades.

VI. CONSIDERAÇÕES FINAIS E PERSPECTIVAS

De maneira geral, este trabalho teve como objetivo principal apresentar as evoluções no estado da arte e da técnica do chamado Paradigma Orientado a Notificações (PON). Tais contribuições para com o PON foram implementadas na forma de ferramentas desenvolvidas sobre C++, de modo a auxiliar os desenvolvedores a construir programas baseados no PON.

A principal motivação para a reconstrução do Framework PON teve origem no trabalho de Banaszewski [15], o qual elaborou a primeira versão do framework para aplicações genéricas. De maneira geral, esta implementação apresentou resultados excelentes em comparação a SBR-Rete para o conjunto de experimentos feitos [15] e resultados bons em comparação a programas do POO (em C++) com certa quantidade de redundância. Em contrapartida, em experimentos subsequentes, a implementação original do framework não demonstrou vantagens satisfatórias ou mesmo demonstrou desvantagens (do ponto de vista de desempenho) em comparação a implementações POO C++ para programas com pouca ou mesmo mediana quantidade de redundâncias.

Em [18][19][20] foi possível constatar que algumas das características que o PON se propunha a otimizar eram realmente consistentes. Entretanto, em relação a comparações com o paradigma predominante POO, as comparações de desempenho, principalmente, não apresentavam os resultados esperados. Neste âmbito, a proposta de otimizações na estrutura do framework como um todo, assim como a proposta de novas estruturas de dados para atuar no processo de notificações pontuais, denominadas de PONLIST, PONVECTOR e PONHASH, apresentaram resultados positivos em relação à implementação anterior.

A estrutura PONVECTOR, particularmente, apresentou os melhores resultados na maioria dos cenários, sendo adotada como estrutura padrão deste novo framework. Os resultados dos experimentos nos estudos de caso apresentaram uma evolução significativa, na qual os tempos de execução apresentaram uma redução média de 50% em relação à versão precedente do framework.

Ainda, em um experimento particular no sistema de controle de vendas foi apresentada a possibilidade de implementações mistas, com o uso da estrutura PONHASH em um caso de elemento ‘sobrecarregado’. Esta implementação composta por entidades mistas apresentou um resultado satisfatório em comparação à mesma aplicação seguindo uma estrutura única. Neste contexto, é importante ressaltar que o desacoplamento entre as entidades PON apresentado nesta nova versão do framework proporcionou ainda mais flexibilidade para a programação neste paradigma, abrindo novos horizontes na busca de otimização da execução de programas.

Em relação a trabalhos futuros, é importante salientar que questões como a identificação de notificações ‘impertinentes’, assim como a otimização pontual para casos específicos de elementos ‘pesados’, podem ser resolvidas em tempo de compilação. Isso simplificaria a tarefa do programador, o qual não precisaria se preocupar com tais questões, no

desenvolvimento de suas aplicações.

Ainda, uma nova versão do framework está sendo proposta herdando a estrutura atual do Framework 2.0. Neste âmbito, a evolução do framework tende a ser menos complexa, uma vez que as suas raízes foram fortemente baseadas nos princípios e padrões de projeto, conforme apresentado na Seção III. A proposta da versão 3.0 do framework é aplicar escalonadores paralelos de entidades (implementados com threads) que trabalhem com a execução paralela das entidades em ambientes *multicore* [21]. Este trabalho se apresenta bastante promissor, pois o paralelismo de execução será feito automaticamente, sem a necessidade da intervenção e conhecimento do programador sobre paralelismo. Ainda, existe um protótipo em desenvolvimento para a distribuição das notificações pela rede por meio do protocolo IP.

Outro avanço importante no âmbito do desenvolvimento de software em PON é a criação de uma linguagem própria, denominada preliminarmente de LingPON [22]. Essa linguagem surgiu principalmente para minimizar o esforço do programador em conhecer toda a infraestrutura do Framework, permitindo o programar em linguagem mais puramente orientada a regras.

Neste âmbito, sinergicamente, um compilador próprio para a linguagem LingPON vem sendo objeto de estudo de pesquisadores do grupo de pesquisa [22]. Os resultados vêm se mostrando bastante positivos, tornando a concepção de programas PON mais natural e facilitada, bem como o desempenho de execução dos programas tem melhorado, reduzindo substancialmente a quantidade de computações e processamento desnecessários, pela eliminação de estruturas de dados [22].

Apesar de apresentar ganhos de desempenho em relação à ferramenta framework, um programa compilado não apresenta o mesmo nível de flexibilidade no âmbito de permitir a criação de novas *Rules* em tempo de execução, possibilitando apenas alterar a estrutura do programa em tempo de compilação. Neste sentido, algum tipo de fusão entre as duas abordagens deve ser proposta para manter as propriedades do PON, bem como proporcionar um desempenho satisfatório.

É importante citar que outros trabalhos sinérgicos vêm estudando a proposta do PON em hardware aplicando um conjunto de lógica reconfigurável, primeiramente na forma de circuitos em PON e, subsequentemente, em forma de um coprocessador em PON [23]. Neste âmbito, uma arquitetura de computador própria para o PON foi proposta e desenvolvida. Esta arquitetura foi denominada de *NOCA (Notification-Oriented Computer Architecture)*. A essência do *NOCA* está organizada como um multiprocessador de granularidade fina executando instruções de forma hierárquica por meio de conjuntos de núcleos especializados [24].

Ainda, embora não mencionado explicitamente neste trabalho, a essência do PON facilita a criação de sistemas complexos, distribuídos e escaláveis. Esta particularidade é interessante para a aplicação na computação ubíqua, na qual múltiplas entidades do PON poderiam estar conectadas em uma rede orientada a notificações, colaborando entre si, de modo a realizar um processo de inferência distribuído e

colaborativo, sem a necessidade de um elemento centralizador. Esta habilidade do PON já está sendo estudada pelos pesquisadores do grupo de pesquisa do PON.

Por fim, neste âmbito, o PON se apresenta como uma alternativa promissora para a composição de sistemas computacionais complexos, distribuídos e escaláveis.

REFERÊNCIAS

- [1] J. M. Simão, P. C. Stadzisz, "Inference Based on Notifications: A Holonic Meta-Model Applied to Control Issues". IEEE Transaction on System, Man, and Cybernetics, Part A V. 9 Issue 1 Pg 238-250, 2009.
- [2] A. F. Ronszcka, R. F. Banaszewski, R. R. Linhares, C. A. Tacla, P. C. Stadzisz, J. M. Simão, "Notification-Oriented and Rete Network Inference: A Comparative Study". In: 2015 IEEE International Conference on Systems, Man and Cybernetics. IEEE SMC 2015, Hong Kong – China. p. 807-814, 2015.
- [3] J. M. Simão, R. F. Banaszewski, C. A. Tacla, P. C. Stadzisz, "Notification Oriented Paradigm (NOP) and Imperative Paradigm: A Comparative Study," Journal of Software Engineering and Applications (JSEA), p.402-416, v.5, n.6, 2012. DOI 10.4236/jsea.2012.56047.
- [4] R. F. Banaszewski, "Paradigma Orientado a Notificações: Avanços e Comparações". Dissertação de Mestrado, Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI) pela Universidade Tecnológica Federal do Paraná (UTFPR). Curitiba, Brazil, Março, 2009. http://files.dirppg.ct.utfpr.edu.br/cpgei/Ano_2009/dissertacoes/Dissertacao_500_2009.pdf
- [5] J. Wyns. Reference Architecture for Holonic Manufacturing Systems – The Key to Support Evolution and Reconfiguration, Ph.D. Thesis, PMA/Katholieke Universiteit Leuven, 1999.
- [6] V. Bako and R. Valette, "Towards a decentralization of rule-based systems controlled by Petri Nets: an application to FMS". 4th Intern. Symposium on Knowledge Engineering. Barcelona, Spain, 1990.
- [7] M. Gabbrielli, S. Martini, "Programming Languages: Principles and Paradigms". Series: Undergraduate Topics in Computer Science. 1st Ed., 2010, XIX, p.440, Softcover. ISBN: 978-1-84882-913-8.
- [8] J. G. Brookshear, "Computer Science: An Overview". Addison Wesley, 2006.
- [9] S. Kaisler, "Software Paradigm", Wiley-Interscience, 1st Edition, 0471483478 John Wiley & Sons, 2005.
- [10] S. W. Loke, "Declarative programming of integrated peer-to-peer and Web based systems: the case of Prolog". Journal of Systems and Software, vol. 79, Issue 4, p. 523-536, 2006. DOI 10.1016/j.jss.2005.04.005.
- [11] J.A. Kang, A. M. K Cheng. "Shortening Matching Time in OPS5 Production Systems". IEEE Transactions on Software Engineering, v. 30, n. 7, p. 448-457, 2004. DOI 10.1109/TSE.2004.32.
- [12] C. L. Forgy, "RETE: A Fast Algorithm for the Many Pattern/Many Object Pattern Match Problem", Artificial Intelligence, n. 19, p. 17-37, North- Holland, 1982. DOI 10.1016/0004-3702(82)90020-0.
- [13] P.-Y. Lee, A. M. Cheng, "HAL: A Faster Match Algorithm". IEEE Transactions on Knowledge and Data Engineering", 14 (5), p. 1047-1058, 2002. DOI 10.1109/TKDE.2002.1033773.
- [14] J. M. Simão, "A Contribution to the Development of a HMS simulation tool and Proposition of a Meta-Model for Holonic Control". Ph. D. Thesis. Graduate School in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal University of Technology - Paraná (UTFPR, Brazil) and Research Center For Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP, France), 2005. http://files.dirppg.ct.utfpr.edu.br/cpgei/Ano_2005/teses/Tese_012_2005.pdf
- [15] R. F. Banaszewski, "Paradigma Orientado a Notificações: Avanços e Comparações". Dissertação de Mestrado, Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI) pela Universidade Tecnológica Federal do Paraná (UTFPR). Curitiba, Brazil, Março, 2009. http://files.dirppg.ct.utfpr.edu.br/cpgei/Ano_2009/dissertacoes/Dissertacao_500_2009.pdf
- [16] G. Z. Valença, "Contribuição para a Materialização do Paradigma Orientado a Notificações via Framework e Wizard". Dissertação de

Mestrado, Programa de Pós-graduação em Computação Aplicada (PPGCA) pela Universidade Tecnológica Federal do Paraná (UTFPR). Curitiba, Brazil, Agosto, 2012. http://repositorio.utfpr.edu.br/jspui/bitstream/1/393/1/CT_PPGCA_M_Valen%C3%A7a,%20Glauber%20Z%C3%A1rate_2012.pdf

- [17] A. F. Ronszcka, "Contribuição para a Concepção de Aplicações no Paradigma Orientado a Notificações (PON) sob o viés de Padrões". Dissertação de Mestrado, Pós-graduação em Engenharia Elétrica e Informática Industrial (CPGEI) pela Universidade Tecnológica Federal do Paraná (UTFPR). Curitiba, Brazil, Agosto, 2012. http://repositorio.utfpr.edu.br/jspui/bitstream/1/327/1/CT_CPGEI_M_%20Ronszcka,%20Adriano%20Francisco_2012.pdf
- [18] G. Z. Valença, R. F. Banaszewski, A. F. Ronszcka, M. V. Batista, R. R. Linhares, J. A. Fabro, P. C. Stadzisz, J. M. Simão, "Framework PON, Avanços e Comparações". In: III Simpósio de Computação Aplicada, SCA 2011, Passo Fundo, RS, Brasil, 2011.
- [19] J. M. Simão, D. L. Belmonte, A. F. Ronszcka, R. R. Linhares, G. Z. Valença, R. F. Banaszewski, J. A. Fabro, C. A. Tacla, P. C. Stadzisz, and M. V. Batista. "Notification Oriented and Object Oriented Paradigm Comparison via Sale System". Journal of Software Engineering and Applications JSEA, Vol. 5 N. 9 2012. DOI 10.4236/jsea.2012.59083.
- [20] J. M. Simão, D. L. Belmonte, G. Z. Valença, M. V. Batista, R. R. Linhares, R. F. Banaszewski, J. A. Fabro, C. A. Tacla, P. C. Stadzisz, and A. F. Ronszcka. "A Game Comparative Study: Object-Oriented Paradigm and Notification-Oriented Paradigm". Journal of Software Engineering and Applications (JSEA), Vol. 5 N. 9, 2012. DOI 10.4236/jsea.2012.59085.
- [21] D. L. Belmonte, R. R. Linhares, P. C. Stadzisz, J. M. Simão. "A new Method for Dynamic Balancing of Workload and Scalability in Multicore Systems". Latin America Transactions, IEEE (Revista IEEE America Latina). Paper aceito em 27/05/2016, 2016.
- [22] C. A. Ferreira, "Linguagem e compilador para o paradigma orientado a notificações (PON): Avanços e comparações". Seminário de Acompanhamento, PPGCA, UTFPR. Curitiba, Brasil, 2014.
- [23] L. F. Pordeus, R. Kerschbaumer, R. R. Linhares, F. A. Witt, P. C. Stadzisz, C. R. E. Lima, J. M. Simão. "Notification Oriented Paradigm to Digital Hardware". Revista SODEBRAS, v. 11, p. 116-122, 2016.
- [24] R. R. Linhares, J. M. Simão, P. C. Stadzisz. "NOCA – A Notification-Oriented Computer Architecture". Latin America Transactions, IEEE (Revista IEEE America Latina), vol 13. No. 5, p. 1593-1604, 2015.



Adriano Francisco Ronszcka received in 2009 his B.Sc. in Information Systems from University of Contestant – Mafra SC, Brazil. He received in 2012 his M.Sc degree in industrial computer science from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He is currently Ph.D. student in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He has experience in Computer Science, with emphasis on Programming Languages, working mainly on the following topics: programming paradigms, programming languages and compilers, design patterns, artificial intelligence and optimization algorithms.



Glauber Zárte Valença received in 2004 his B.Sc. in computer science from State University of Western Paraná, and in 2008 concluded his specialization in architecture and development in Java technology from Federal University of Technology - Paraná (UTFPR), Brazil. He received in 2012 his M.Sc degree in computer science from Graduate Program in Applied Computing (PPGCA), UTFPR, Brazil. He has experience in Computer Science, with emphasis on software engineering, analysis and development of embedded systems and web applications.



Robson Ribeiro Linhares received in 1999 his B.Sc. in Electrical Engineering, with emphasis on Electronics and Telecommunications, from Federal University of Technology - Paraná (UTFPR), Brazil, which is formerly known as Federal Center for Technological Education of Parana. He received in 2001 his M.Sc degree in software engineering for real-time systems from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil. He received in 2015 the Ph.D. degree in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, as well as

Professor with the UTFPR, teaching in a set of educational programs such as the undergraduate courses of Computer Engineering and Information Systems and the Graduate Program in Applied Computing (PPGCA). His research interests include software engineering, programming languages and paradigms, embedded systems and computer architectures.



João Alberto Fabro received in 1994 his B.Sc. in informatics from Federal University of Paraná (UFPR), Brazil, and in 1996 the M.Sc. degree in electrical engineering from UNICAMP, Campinas SP, Brazil. He received in 2003 the Ph.D. degree in computer engineering from Graduate Program in Electrical Engineering and Industrial Informatics (CPGEI), UTFPR, Brazil, and Postdoctoral degree at the Faculty of Engineering of the University of Porto, Portugal. He was a professor at UNIOESTE (State University of Western Paraná) from 1998 to 2007. Since 2008 he is Professor at Federal University of Technology - Paraná (UTFPR) teaching in a set of educational programs. He has experience in Computer Science, with emphasis on Artificial Intelligence, working mainly in the following subjects: Autonomous Mobile Robotics, Computational Intelligence (Neural Networks, Fuzzy Systems, Evolutionary Computing) and Educational Technologies.



Paulo César Stadzisz received in 1987 the Diploma in Data Processing Technologies from the Federal University of Paraná (UFPR), Brazil, and in 1990 the M.Sc. degree in industrial computer science from Graduate Program in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal Center for Technological Education of Parana (CEFET-PR). He received in 1997 the Ph.D. degree from the Franche-Comté University (France). He is currently Professor at Federal University of Technology - Paraná (UTFPR) teaching in a set of educational programs. He namely teaches and coordinates researches in the CPGEI/UTFPR doctoral program associated to CNPq (National Council for Scientific and Technological Development) and CAPES (Coordination for the Improvement of Higher Education Personnel) of Brazil. His publications and researches include systems modeling and analysis, simulation, and software engineering.



Jean Marcelo Simão received in 1994 the Technical degree in informatics from State School of Paraná (CEP) and in 1998 the B.Sc. degree in computer science from State University of Ponta Grossa (UEPG). In 2001, he obtained the M.Sc. degree from Graduate School in Electrical Engineering and Industrial Computer Science (CPGEI) at Federal Center for Technological Education of Parana (CEFET-PR), situated in Curitiba (Brazil), nowadays named as Federal University of Technology - Paraná (UTFPR). On June 2005, he obtained the Ph.D. degree, after a double thesis, in the domains of industrial computer science at CPGEI/UTFPR and computer engineering & automatics at Research Center for Automatic Control of Nancy (CRAN) - Henry Poincaré University (UHP), situated in France. After, in 2006, he developed teaching activities in a Master at UHP and researches ones at CRAN in a post-doctoral context. Nowadays, he is Associated Professor at UTFPR. His teaching activities concern computer science to engineering courses, whereas his publications and current research interests include agile/holonic manufacturing, artificial intelligence, software/system engineering, computer science, and programming and computer paradigms.